🎭 Playwright Assertions

> *A quick reference guide to Playwright Assertions (TypeScript)*

✅ Auto-Retrying Assertions

What are Auto-Retrying Assertions?

Playwright automatically retries these assertions until:

- ✅ The assertion passes
- ⏱️ The assertion timeout is reached (Default: **5 seconds**).During this time, Playwright will continuously poll and re-test the element (every 100ms) until the expected condition is met or the 5-second window expires. If it expires without passing, the test throws a TimeoutError.

Benefits

- 🔄 Reduces flaky tests
- 🚫 No need for manual waits (`waitForTimeout()`)
- ⚡ Ideal for modern asynchronous web applications
- ✅ Must be used with `await`

```
await expect(locator).toBeVisible();
```

> ***Default Assertion Timeout:*** *`5 seconds`*

You can customize it globally:

```
// playwright.config.ts
  export default defineConfig({
    expect: {
      timeout: 10000
    }
  });
```

Visibility Assertions

Assertion	Description	Usage Example
`await expect(locator).toBeAttached()`	Verifies that the element is attached to the DOM.	`await expect(page.locator('#loginBtn')).toBeAttached();`
`await expect(locator).toBeVisible()`	Verifies that the element is visible on the page.	`await expect(page.getByRole('button', { name: 'Login' })).toBeVisible();`
`await expect(locator).toBeHidden()`	Verifies that the element is hidden or not visible.	`await expect(page.locator('.loading')).toBeHidden();`
`await expect(locator).toHaveAccessibleDescription()`	Verifies the accessible description of the element.	`await expect(page.locator('#email')).toHaveAccessibleDescription('Enter your email');`
`await expect(locator).toHaveAccessibleName()`	Verifies the accessible name of the element.	`await expect(page.getByRole('button')).toHaveAccessibleName('Submit');`

State Assertions

Assertion	Description	Usage Example
`await expect(locator).toBeChecked()`	Verifies that a checkbox or radio button is checked.	`await expect(page.locator('#remember')).toBeChecked();`
`await expect(locator).toBeDisabled()`	Verifies that the element is disabled.	`await expect(page.locator('#submit')).toBeDisabled();`
`await expect(locator).toBeEditable()`	Verifies that the element is editable.	`await expect(page.locator('#username')).toBeEditable();`
`await expect(locator).toBeEnabled()`	Verifies that the element is enabled.	`await expect(page.locator('#submit')).toBeEnabled();`
`await expect(locator).toBeFocused()`	Verifies that the element currently has keyboard focus.	`await expect(page.locator('#search')).toBeFocused();`

📝 Text Assertions

Assertion	Description	Usage Example
`await expect(locator).toContainText()`	Verifies that the element contains the expected text.	`await expect(page.locator('.message')).toContainText('Success');`
`await expect(locator).toHaveText()`	Verifies that the element exactly matches the expected text.	`await expect(page.locator('h1')).toHaveText('Dashboard');`
`await expect(locator).toHaveValue()`	Verifies the value of an input field.	`await expect(page.locator('#email')).toHaveValue('admin@test.com');`
`await expect(locator).toHaveValues()`	Verifies selected values in a multi-select element.	`await expect(page.locator('#countries')).toHaveValues(['India','USA']);`

🏷️ Attributes & Properties

Assertion	Description	Usage Example
`await expect(locator).toContainClass()`	Verifies that the element contains one or more CSS classes.	`await expect(page.locator('#status')).toContainClass('active');`
`await expect(locator).toHaveClass()`	Verifies the exact CSS class.	`await expect(page.locator('#status')).toHaveClass('badge badge-success');`
`await expect(locator).toHaveAttribute()`	Verifies a DOM attribute.	`await expect(page.locator('#email')).toHaveAttribute('type','email');`
`await expect(locator).toHaveCSS()`	Verifies a CSS property value.	`await expect(page.locator('button')).toHaveCSS('color','rgb(255, 255, 255)');`
`await expect(locator).toHaveId()`	Verifies the element ID.	`await expect(page.locator('input')).toHaveId('username');`

📊 Count Assertions

Assertion	Description	Usage Example
`await expect(locator).toHaveCount()`	Verifies the number of matching elements.	`await expect(page.locator('.product')).toHaveCount(10);`

♿ Role Assertions

Assertion	Description	Usage Example
`await expect(locator).toHaveRole()`	Verifies the ARIA role.	`await expect(page.getByRole('button')).toHaveRole('button');`

🌐 Page Assertions

Assertion	Description	Usage Example
`await expect(page).toHaveTitle()`	Verifies the page title.	`await expect(page).toHaveTitle('Dashboard');`
`await expect(page).toHaveURL()`	Verifies the current page URL.	`await expect(page).toHaveURL('https://example.com/dashboard');`

🌍 API Assertions

Assertion	Description	Usage Example
`await expect(response).toBeOK()`	Verifies that the API response status is successful (2xx).	`await expect(response).toBeOK();`

🚫 Non-Retrying Assertions

> *These assertions **do NOT retry** automatically.*

⚠️ Since web pages update asynchronously, excessive use of non-retrying assertions can make tests flaky.

🚫 Value Assertions

Assertion	Description	Usage Example
`expect(value).toBe()`	Verifies that two primitive values are exactly equal using `Object.is()`.	`expect(status).toBe('Success');`
`expect(value).toEqual()`	Verifies deep equality of objects, arrays, and nested values.	`expect(user).toEqual(expectedUser);`
`expect(value).toStrictEqual()`	Verifies strict deep equality, including object types and `undefined` properties.	`expect(data).toStrictEqual(expectedData);`
`expect(value).toBeDefined()`	Verifies that the value is defined (not `undefined`).	`expect(token).toBeDefined();`
`expect(value).toBeUndefined()`	Verifies that the value is `undefined`.	`expect(result).toBeUndefined();`
`expect(value).toBeNull()`	Verifies that the value is `null`.	`expect(response).toBeNull();`
`expect(value).toBeTruthy()`	Verifies that the value evaluates to `true` in a boolean context.	`expect(isLoggedIn).toBeTruthy();`
`expect(value).toBeFalsy()`	Verifies that the value evaluates to `false` in a boolean context.	`expect(hasError).toBeFalsy();`
`expect(value).toBeNaN()`	Verifies that the value is `NaN` (Not-a-Number).	`expect(Number('abc')).toBeNaN();`

🔢 Numeric Assertions

Assertion	Description	Usage Example
`expect(value).toBeCloseTo()`	Verifies that two floating-point numbers are approximately equal.	`expect(price).toBeCloseTo(99.99, 2);`
`expect(value).toBeGreaterThan()`	Verifies that a value is greater than the expected value.	`expect(totalItems).toBeGreaterThan(10);`
`expect(value).toBeGreaterThanOrEqual()`	Verifies that a value is greater than or equal to the expected value.	`expect(score).toBeGreaterThanOrEqual(80);`
`expect(value).toBeLessThan()`	Verifies that a value is less than the expected value.	`expect(age).toBeLessThan(60);`
`expect(value).toBeLessThanOrEqual()`	Verifies that a value is less than or equal to the expected value.	`expect(cartItems).toBeLessThanOrEqual(20);`

📚 Collection & String Assertions

Assertion	Description	Usage Example
`expect(value).toContain()`	Verifies that a string, array, or set contains the expected value.	`expect(tags).toContain('Admin');`
`expect(array).toContainEqual()`	Verifies that an array contains an object with matching values.	`expect(users).toContainEqual({ id: 1, name: 'John' });`
`expect(value).toHaveLength()`	Verifies the length of a string, array, or collection.	`expect(password).toHaveLength(8);`
`expect(object).toHaveProperty()`	Verifies that an object contains the specified property.	`expect(user).toHaveProperty('email');`
`expect(string).toMatch()`	Verifies that a string matches a regular expression.	`expect(email).toMatch(/@gmail\.com$/);`
`expect(object).toMatchObject()`	Verifies that an object contains the expected properties and values.	`expect(user).toMatchObject({ role: 'Admin' });`
`expect(value).toBeInstanceOf()`	Verifies that an object is an instance of a specific class.	`expect(error).toBeInstanceOf(Error);`

🎯 Asymmetric Matchers

Useful for flexible matching inside other assertions.

Matcher	Description	Usage Example
`expect.any()`	Matches any value of the specified type.	`expect(user).toEqual({ id: expect.any(Number) });`
`expect.anything()`	Matches any value except `null` and `undefined`.	`expect(value).toEqual(expect.anything());`
`expect.arrayContaining()`	Verifies that an array contains the specified elements.	`expect(colors).toEqual(expect.arrayContaining(['Red', 'Blue']));`
`expect.arrayOf()`	Verifies that every element in an array matches the specified matcher.	`expect(ids).toEqual(expect.arrayOf(expect.any(Number)));`
`expect.closeTo()`	Matches a number that is approximately equal to the expected value.	`expect(product).toEqual({ price: expect.closeTo(99.99, 2) });`
`expect.objectContaining()`	Verifies that an object contains the specified properties.	`expect(user).toEqual(expect.objectContaining({ name: 'John' }));`
`expect.stringContaining()`	Verifies that a string contains the expected substring.	`expect(message).toEqual(expect.stringContaining('Success'));`
`expect.stringMatching()`	Verifies that a string matches the specified regular expression.	`expect(message).toEqual(expect.stringMatching(/^Welcome/));`

❌ Negating Assertions

Use `.not` to verify the opposite condition.

```
expect(value).not.toEqual(0);


  await expect(locator)
        .not.toContainText("Admin");


  await expect(locator)
        .not.toBeVisible();
```

□ Soft Assertions

Normally, a failed assertion stops test execution.

Soft assertions continue execution while marking the test as failed.

```
await expect.soft(locator).toHaveText("Welcome");

  console.log("This line still executes.");
```

Use Soft Assertions When

- ✅ Collecting multiple failures
- ✅ End-to-end validation
- ✅ Form verification
- ✅ UI regression testing

💡 Best Practices

✅ Prefer **Auto-Retrying Assertions**

✅ Avoid manual waits

✅ Use locator assertions whenever possible

✅ Use `.not` for negative validations

✅ Use `expect.soft()` for multiple independent checks

✅ Keep assertion timeout reasonable

📌 Quick Summary

Category	Common Assertions
👁️ Visibility	`toBeVisible()`, `toBeHidden()`, `toBeAttached()`
⚙️ State	`toBeEnabled()`, `toBeDisabled()`, `toBeChecked()`
📝 Text	`toHaveText()`, `toContainText()`
🏷️ Attributes	`toHaveAttribute()`, `toHaveClass()`, `toHaveCSS()`
📊 Count	`toHaveCount()`
🌐 Page	`toHaveTitle()`, `toHaveURL()`
🌎 API	`toBeOK()`
🚫 Negative	`.not`
☐ Soft	`expect.soft()`

🚀 Pro Tip

Prefer Playwright's built-in auto-retrying assertions over manual waits. They make your tests faster, more stable, and significantly less flaky.